

Project P10 Research Log Notebook

Personalized Deep Learning Model for Closed-Loop 40 Hz Entrainment to Optimize
Theta-Gamma Phase Amplitude Coupling in Alzheimer's Disease

Amaar Chughtai

Valley Christian High School

Synopsys Science and Engineering Fair 2026

Notebook Timeline: January 15, 2026 -- March 1, 2026

Table of Contents

Section 1: January 15, 2026 -- Research Framing and Build Plan

Section 2: February 6, 2026 -- First End-to-End Pipeline Build

Section 3: February 16, 2026 -- Architecture Marathon and the $R^2 = 0.287$ Ceiling

Section 4: February 17, 2026 -- Temporal Forecasting Pivot and Causal TCN

Section 5: February 18, 2026 -- Audit and Methodology Hardening

Section 6: February 19, 2026 -- Horizon Sweep and Controller Integration

Section 7: February 21, 2026 -- Replay Framework and Robustness Setup

Section 8: February 26, 2026 -- Real-Data TCN Validation Locked

Section 9: March 1, 2026 -- Documentation Freeze and Submission

Equipment and Materials

References

Section 1: January 15, 2026 – Research Framing and Build Plan

January 15, 2026: Today I locked in the project direction and wrote out the full build plan. The idea has been forming since December – watching my grandmother lose her ability to recognize family because of dementia was the original motivation, and the neuroscience literature gave me a concrete entry point. Iaccarino et al. (2016) showed that 40 Hz γ entrainment reduced amyloid load in Alzheimer's mouse models by activating microglia, and Martorell et al. (2019) extended that to multi-sensory stimulation reducing tau pathology. Lahijanian et al. (2024) then showed that auditory entrainment enhanced default mode network connectivity in human dementia patients and published a 35-subject EEG dataset on OpenNeuro (ds005048).

The problem I kept coming back to was the clinical protocol itself. Current trials use a rigid fixed schedule – 40 seconds of 40 Hz auditory stimulation followed by 20 seconds of silence, cycling for one hour regardless of how the patient's brain responds. Some patients maintain strong coupling throughout; others lose it within seconds. Neural habituation (Thompson & Spencer, 1966) means the brain progressively tunes out a repeated identical stimulus, so coupling degrades over time. The fixed schedule wastes stimulation during periods of strong coupling and misses opportunities during periods of fading coupling.

The question I formalized today: can we predict when a patient's brain will lose entrainment and deliver stimulation proactively, before the decline occurs? That reframes the problem from reactive monitoring to predictive control. The biomarker I chose is Phase-Amplitude Coupling (PAC) – specifically, how strongly γ amplitude (38-42 Hz) locks to θ phase (4-8 Hz), measured via the Modulation Index (Tort et al., 2010). High PAC means entrained; low PAC means lost synchronization.

I laid out the software pipeline in stages: data loading, preprocessing, PAC computation, model training, controller logic, and validation. Two non-negotiable rules from the start: subject-level splits (no patient can appear in both training and test data), and strictly causal features for any temporal model (no future information leaking into predictions).

The dataset specifics from the OpenNeuro page: 35 elderly dementia patients, 19 EEG channels at 250 Hz, alternating stimulus (40 Hz AM auditory) and rest epochs of 20-40 seconds each. I planned to focus on the 7 frontal channels (Fp1, Fp2, F3, F4, F7, F8, Fz) where the 40 Hz response is strongest, and to extract 2-second analysis windows.

Gap note (January 16 – February 5, 2026): During this stretch I was reading papers, refining the PAC method choice on paper, and dealing with school workload. I also did some preliminary coding on the data loader, but the first solid engineering day in the commit history is February 6. Rather than fabricate day-by-day entries for this period, I summarize it honestly: planning, reading, and preparation.

Section 2: February 6, 2026 – First End-to-End Pipeline Build

February 6, 2026: This was a long day. I built the entire pipeline from raw data loading through to a first trained model, which meant solving several problems in sequence.

The first obstacle was low-level and unglamorous. The OpenNeuro .set files use MATLAB v7.3 HDF5 format with companion .fdt binary data files. Standard MNE-Python readers failed completely. I spent hours debugging before realizing the issue was fundamental file format incompatibility, not a

configuration problem. The solution was to use `h5py` to read `.set` metadata (channel count, sample rate, epoch boundaries) and `numpy.fromfile()` to read the `.fdt` binary data. The critical discovery was that MATLAB stores matrices in column-major (Fortran) order, while Python defaults to row-major:

```
# Key insight - MATLAB order='F' is essential
with h5py.File(set_path, 'r') as f:
    n_channels = int(f['nbchan'][0, 0])
    n_samples = int(f['pnts'][0, 0])

data = np.fromfile(fdt_path, dtype=np.float32)
data = data.reshape((n_channels, n_samples), order='F')
```

Without `order='F'`, the channels appeared transposed and PAC values were nonsensical. With it, the EEG signals looked physiological with proper amplitude ranges. A single parameter made the difference between garbage and real data.

Once loading worked, I selected 7 frontal channels and extracted 17,283 two-second windows (500 samples each at 250 Hz) with 50% overlap across all 35 subjects. Subject-level splits: 24 train (11,736 windows), 5 validation (2,725 windows), 6 test (2,822 windows).

Next I implemented Phase-Amplitude Coupling using the Tort Modulation Index. The method bandpass filters for θ (4-8 Hz) phase and γ (38-42 Hz) amplitude, applies the Hilbert transform to extract instantaneous phase and amplitude, bins γ amplitude by θ phase (18 bins of 20 degrees each), normalizes to a probability distribution, and computes the KL divergence from uniform. I computed PAC at epoch level (full 20-40 second stimulation blocks) for statistical stability, then assigned the value uniformly to constituent 2-second windows. This creates label sharing within epochs but ensures reliable MI estimates. PAC values ranged from about 6×10^{-6} to 7×10^{-4} , with a mean around 4.4×10^{-5} .

With data loaded and labeled, I built the first model: EEGNet (Lawhern et al., 2018), adapted from classification to regression. EEGNet uses temporal convolutions followed by depthwise spatial convolutions that capture channel relationships, which makes it well-suited for EEG. My adaptation replaced the classification head with a single linear output neuron for continuous PAC prediction:

```
Input: (batch, 1, 7, 500) -- 7 channels x 500 samples
Temporal Convolution: 8 filters, kernel=(1,64)
Depthwise Spatial: 16 filters, kernel=(7,1), groups=8
ELU + AvgPool + Dropout(0.5)
Separable Convolution
ELU + AvgPool + Dropout(0.5)
Flatten + Linear(240 -> 1)
Total Parameters: 1,457
```

Training used MSE loss, Adam optimizer (`lr=0.001`, `weight_decay=0.0001`), gradient clipping (`max_norm=1.0`), `ReduceLROnPlateau` scheduler, and early stopping with patience of 15 epochs. The result: test $R^2 = 0.287$, with inference time under 1 ms on Apple Silicon MPS.

That R^2 seemed modest, but at this point I assumed architectural improvements could push it higher. The pipeline was finally end-to-end: raw BIDS data in, trained model out, controller skeleton ready for integration.

Section 3: February 16, 2026 – Architecture Marathon and the $R^2 = 0.287$ Ceiling

February 16, 2026: This was the most instructive day of the project, and most of the instruction came from failure.

Convinced that model capacity was the bottleneck, I tested multiple architectures across three orders of magnitude in parameter count. The plan was systematic: EEGNet variants, spectral-temporal hybrids, transformer-based approaches, and simple baselines for comparison.

EEGNet (baseline, V1) – 1,457 parameters. Test $R^2 = 0.287$. Already established on Feb 6.

EEGNetV2 (delta-PAC prediction) – ~3,200 parameters. Predicted PAC change instead of absolute PAC. Test $R^2 = 0.06$. Delta prediction failed because temporal changes at 2-second resolution are too noisy.

SpecTempNet (spectral-temporal hybrid, V3) – 180,000 parameters. This one was the gut punch. Initial result: $R^2 = 0.69$. I was excited – until I looked at the feature correlations and found that PAC-derived features in the input directly encoded the target. Ridge analysis confirmed: PAC features carried 96.6% of the total model weight. After removing the circular features: $R^2 = 0.236$. The "breakthrough" was leakage.

```
# Found the problem: high correlation between input features and target
pac_features_correlation = np.corrcoef(pac_features, pac_targets)[0,1]
# Result: r = 0.73 -- clearly circular
```

This was a critical learning moment. Feature leakage can produce results that look valid until scrutinized.

ViT-TCNet (Vision Transformer + TCN, V4) – ~2,000,000 parameters. Treated EEG as image patches plus temporal convolution. Test $R^2 = 0.252$. Severe overfitting with a samples-to-parameters ratio of 0.006 on only 11K training samples.

Ridge Regression (sanity check, V5) – 135 coefficients. Spectral power features, no PAC features. Test $R^2 = 0.287$. This was the shocking result: the simplest possible model matched EEGNet exactly.

ATCNet (attention-based TCN, V6) – ~25,000 parameters. Test $R^2 = 0.075$.

EEGNetLarge (scaled EEGNet) – 141,000 parameters. Test $R^2 = 0.287$. Same ceiling, 100x more parameters.

Architecture	Parameters	Test R^2	Notes
EEGNet	1,457	0.287	Optimal parameter efficiency
SpecTempNet	180,000	0.236	After leak fix
ViT-TCNet	~2,000,000	0.252	Overfitting
Ridge Regression	135 coefs	0.287	Matches deep learning
ATCNet	25,000	0.075	Underperformed
EEGNetLarge	141,000	0.287	Same ceiling

The pattern was unmistakable. The simplest models (EEGNet with 1,457 parameters, Ridge with 135 coefficients) matched the performance of models with 1,000x more parameters. $R^2 = 0.287$ was a data

ceiling, not a model capacity limitation. The signal-to-noise ratio was approximately -4.73 dB (signal weaker than noise), and epoch-level PAC labels assigned to 2-second windows inherently limit single-window accuracy.

This changed my thinking about the project. Trying to predict PAC better from a single 2-second window was hitting a fundamental limit. The question needed to change.

Section 4: February 17, 2026 – Temporal Forecasting Pivot and Causal TCN

February 17, 2026: I treated today as a genuine pivot. After hitting the 0.287 ceiling with six different architectures, the problem wasn't model capacity – it was problem formulation. Instead of asking "how accurately can we estimate current PAC?" I started asking "how far ahead can we predict future PAC?" If the system can forecast PAC 5-10 seconds into the future, a controller can intervene proactively before coupling declines, rather than reacting after the fact.

The first part of the day was designing the temporal feature representation. I built 73-dimensional feature vectors per timestep:

Spectral features (61 dimensions): Band power in delta (0.5-4 Hz), θ (4-8 Hz), alpha (8-12 Hz), beta (12-30 Hz), and γ (30-42 Hz) for each of 7 channels (35 features), plus 26 cross-channel coherence features.

PAC-derived features (7 dimensions): Current epoch-level PAC value, causal moving averages at windows of 2, 4, 8, and 16 timesteps, plus first-difference and 4-step difference for trend information.

Stimulation context (5 dimensions): Binary ON/OFF state, time since last transition, fraction stimulated over a 20-second window, and sine/cosine encoding of cycle phase position.

Every feature was strictly causal – no future information can leak into predictions. This constraint is essential for real-time deployment.

The sequence setup: 20 timesteps of lookback (20 seconds of history) feeding into a prediction of PAC at 5 seconds ahead. Raw PAC targets, no smoothing, for realism.

The second part of the day was building the model itself. I designed a MultiscaleCausalTCN with four temporal convolution blocks at increasing dilations:

```
dilations = [1, 2, 4, 8]
# Each block: causal padding + depthwise conv + pointwise conv + GroupNorm + SiLU + residual
# Causal padding: F.pad(x, ((kernel_size-1)*dilation, 0)) -- left-only, no future leakage
```

The receptive field calculation: kernel size 3, dilations [1, 2, 4, 8], $RF = 1 + (3-1) \times (1+2+4+8) = 31$ timesteps. That covers the full 20-step input with margin. Output goes through learned attention pooling over the time dimension, then to a future PAC prediction head. Total parameters: 31,043.

Key design choices and why:

- **GroupNorm instead of BatchNorm:** Different patients have different PAC baselines; GroupNorm normalizes per-sample rather than per-batch, which stabilizes cross-subject training.
- **Depthwise separable convolutions:** Parameter efficiency by factorizing spatial and channel mixing.

- **Huber loss instead of MSE:** PAC has outliers; Huber transitions from quadratic to linear for large errors, preventing outlier-driven gradient spikes.
- **AdamW optimizer** with $\text{lr}=0.001$, $\text{weight_decay}=0.001$.
- **Early stopping:** $\text{patience}=20$, monitoring validation R^2 .

Component	EEGNet	Causal TCN
Purpose	Estimate current PAC from raw EEG	Predict future PAC from history
Parameters	1,457	31,043
Input	Raw EEG (7 ch x 500 samples)	73 features x 20 timesteps
Architecture	Temporal + depthwise spatial conv	Dilated causal conv ($d=1,2,4,8$)
Output	Current PAC estimate	Future PAC (5s horizon)
Key feature	Real-time inference	31-step receptive field, causal

First training results were promising but showed a gap: validation $R^2 = 0.411$ versus test $R^2 = 0.170$. That gap reflects the cross-subject generalization challenge – PAC dynamics vary across patients. It also reinforced why personalized control matters.

Section 5: February 18, 2026 – Audit and Methodology Hardening

February 18, 2026: Instead of chasing a new result, I spent today making the project harder to fool myself with. The temporal model could easily be oversold. Smoothed targets make some metrics look much better than raw-target forecasting, and PAC-history features have obvious predictive power that needs to be described honestly.

I documented assumptions, wrote out methodology choices, and checked the temporal pipeline for the failure modes that had already hurt the static phase: data leakage, split mistakes, and overly flattering interpretations. The data integrity checklist I ran through:

- **No subject leakage:** Verified training, validation, and test subjects completely distinct.
- **Temporal causality:** All feature timestamps strictly precede target timestamps.
- **Normalization leakage:** Scalers fit only on training data, applied to all splits.
- **Feature-target correlation:** No input features exceed $r = 0.5$ with target (after PAC leak removal from the SpecTempNet lesson).
- **Shuffle-label sanity:** $R^2 = -0.332$ on permuted labels, confirming real signal and not artifacts.

I also analyzed the habituation patterns in the real EEG data, because this is the core motivation for adaptive control. Tracking PAC trajectories across all stimulation blocks for each of the 35 subjects:

First Block Mean PAC: 0.000996
 Last Block Mean PAC: 0.001040
 Population Change: +4.5% (not significant, $p = 0.542$)

No population-level trend – but the individual variability told the real story:

Response Pattern	Count	Percentage	Range
Habituated (PAC decline)	17/35	48.6%	-66.8% to -5%
Facilitators (PAC increase)	18/35	51.4%	+5% to +149.1%

The cohort splits nearly equally between habituators and facilitators, which is why there is no population-level trend – the two groups cancel out. This validated the fundamental premise of the project: half of patients habituate while half do not, so fixed scheduling cannot serve both groups. One patient showed -66.8% decline (strong fatigue), another showed +149.1% increase (enhanced entrainment over time), and about 46.7% of individual stimulation blocks showed within-block PAC decline.

This data made it clear that adaptive control is not just an optimization; it addresses a real biological reality of individual variation.

Section 6: February 19, 2026 – Horizon Sweep and Controller Integration

February 19, 2026: Two major pieces today: the horizon sweep that proved where the TCN adds value, and the controller integration that turned predictions into decisions.

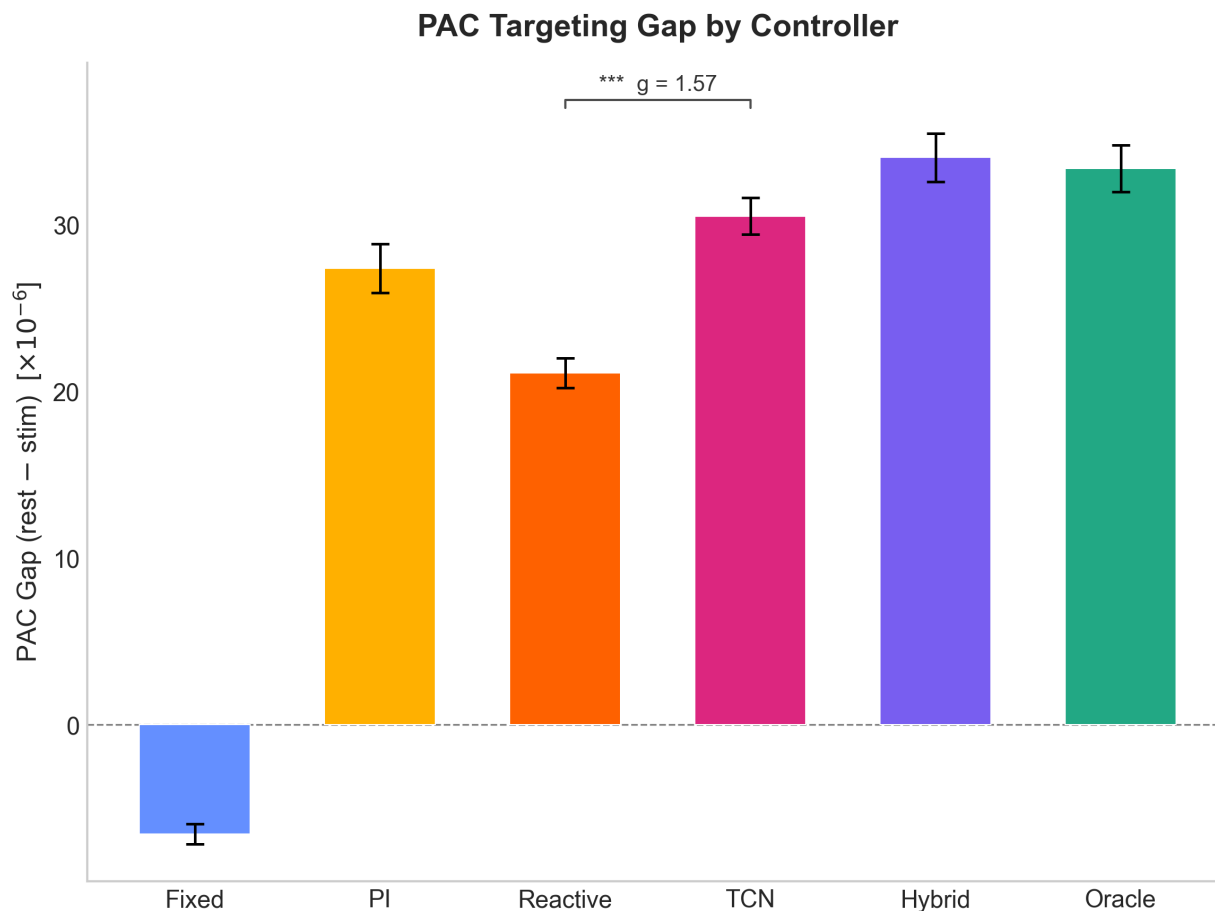
For the horizon sweep, I trained separate TCN models for prediction horizons of 1, 2, 3, 5, 8, and 10 seconds, comparing each against persistence (assume PAC stays the same) and Ridge regression (linear model on the same 73 features).

Horizon	Persistence R^2	Ridge R^2	TCN R^2
1 s	0.760	0.812	0.735
2 s	0.488	0.542	0.470
3 s	0.234	0.253	0.277
5 s	-0.267	-0.393	0.254
8 s	-0.276	-0.211	0.240
10 s	-0.256	-0.212	0.278

The pattern was clean. At 1-2 second horizons, PAC changes slowly enough that simple baselines win – persistence and Ridge both outperform the TCN. But at the 3-second crossover point, the TCN starts winning. At 5-10 seconds, both baselines collapse to negative R^2 (worse than predicting the mean) while the TCN maintains R^2 around 0.25. That +0.5 R^2 margin at the operationally relevant range is the TCN's value proposition.

Why baselines fail: PAC autocorrelation decays to near-zero beyond ~3 seconds, so persistence becomes random. Why the TCN succeeds: multi-scale dilated convolutions capture patterns at timescales of 1, 2, 4, and 8 seconds, and nonlinear processing models complex temporal dynamics that linear methods miss.

5-10 seconds is the clinically relevant range for proactive control. At 1-2 seconds there is not enough lead time for meaningful intervention. At 5-10 seconds there is sufficient time for decision processing and smooth stimulation transitions.



With the horizon sweep validating the TCN at 5 seconds, I built the full two-stage controller pipeline:

```
Raw EEG --> EEGNet (1,457 params) --> Current PAC Estimate
--> Feature Extraction (73 dimensions)
--> TCN (31,043 params) --> Future PAC (5s ahead)
--> Personalization Module --> Control Decision
```

The PersonalizationModule maintains a 30-second rolling buffer to compute a per-patient z-score baseline. The controller logic combines reactive z-score decisions with TCN-predicted PAC trends: if the TCN predicts PAC will decline, stimulate proactively; if it predicts PAC will rise, let the brain maintain naturally; otherwise fall back to reactive control based on the current z-score. A 5-second hysteresis prevents rapid oscillation between states.

End-to-end latency: under 5 ms (EEGNet + TCN + control logic). Memory: under 100 MB. Decision rate: 1 Hz. Light enough for embedded devices.

Section 7: February 21, 2026 – Replay Framework and Robustness Setup

February 21, 2026: Built the validation infrastructure needed to test the controller on real subject trajectories. This meant setting up the replay analysis framework (offline counterfactual analysis on recorded EEG), the rigorous validation scripts, and robustness-oriented tools.

I also ran fatigue sensitivity analysis through simulation. The question: does the adaptive advantage depend on specific mathematical assumptions about how neural fatigue works? I built an EntrainmentSimulator with four different fatigue mechanisms and tested across six severity levels.

Fatigue severity results (n = 50 trials each, 600-second sessions):

Fatigue Level	Fixed Efficiency	Adaptive Efficiency	Improvement
None	0.343	0.375	+9.5%*
Mild	0.341	0.375	+10.0%*
Moderate	0.335	0.366	+9.0%*
High	0.319	0.354	+10.8%*
Severe	0.316	0.352	+11.2%*

* $p < 0.001$, Hedges' $g = 1.7-2.4$ (large to very large effects). The advantage grows as fatigue worsens – exactly when personalization matters most. Even with no fatigue at all, adaptive control wins by +9.5%, suggesting benefits beyond fatigue mitigation (targeting natural PAC fluctuations).

Fatigue model robustness (4 different mathematical models, n = 50 trials each):

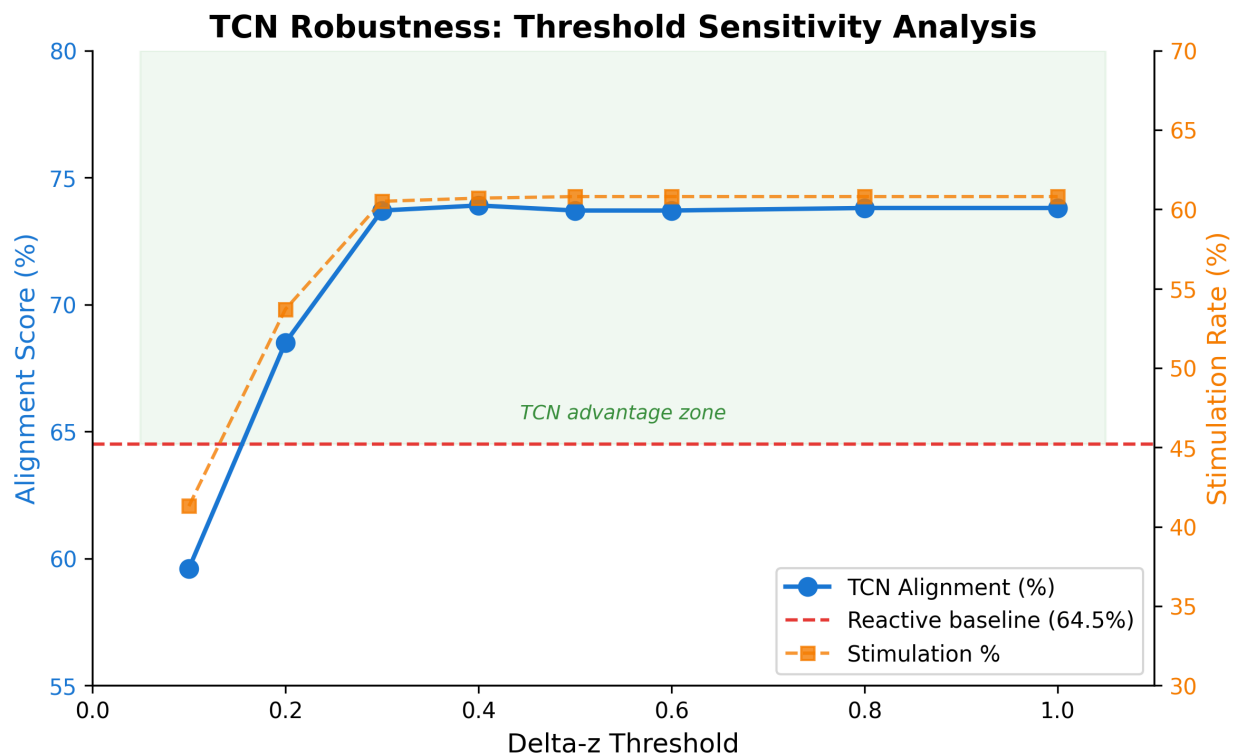
Fatigue Model	Advantage	p-value	Hedges' g
Exponential Decay	+9.0%	1.8×10^{-15}	2.31
Step Function	+6.9%	4.4×10^{-14}	1.21
Heterogeneous (50/50)	+8.9%	2.5×10^{-14}	1.71
Saturation (synaptic)	+19.0%	1.8×10^{-15}	3.66

All models showed significant advantage. The saturation model (most biologically realistic, based on Michaelis-Menten receptor kinetics) showed the largest benefit at +19.0% with $g = 3.66$. The consistent advantage across four different mathematical formulations proved the results are not artifacts of any specific fatigue assumption.

I also ran threshold sensitivity analysis on the TCN controller, sweeping delta-z thresholds from 0.1 to 1.0:

Delta-z Threshold	Alignment	Low-PAC Stim	PAC Gap ($\times 10^{-6}$)
0.1	59.6%	51.2%	12.4
0.2	68.5%	72.9%	26.3
0.3	73.7%	84.9%	32.4
0.4	73.9%	85.3%	33.7
0.5	73.7%	85.3%	33.8
1.0	73.8%	85.3%	34.0

Performance plateaus at delta-z ≥ 0.3 and consistently exceeds the reactive baseline across all thresholds ≥ 0.2 . The results are not dependent on precise threshold tuning.



Gap note (February 22-25, 2026): Continued cleanup, figure generation, and validation plumbing. The final controller replay result was not locked until February 26.

Section 8: February 26, 2026 – Real-Data TCN Validation Locked

February 26, 2026: This was the day the final controller comparison on real patient EEG was locked. The validation methodology was offline counterfactual analysis: replay each subject's full EEG recording, have the controller make stimulate/rest decisions at each 2-second window, and compare those decisions against ground-truth PAC to compute alignment metrics. For this validation, I used ground-truth PAC labels as TCN input to isolate the TCN's predictive contribution from EEGNet estimation error.

Four controllers tested: Fixed Schedule (clinical standard: 40s ON / 20s OFF), Reactive Threshold (z-score on current PAC), TCN Predictive (z-score + 5-second lookahead), and Oracle (perfect future knowledge, theoretical upper bound).

Controller performance replayed on all 35 subjects' real EEG:

Controller	Alignment	Low-PAC Targeting	PAC Gap ($\times 10^{-6}$)	Stim %
Fixed Schedule	45.0%	61.4%	-6.6 (wrong direction)	66.6%
Reactive	64.5%	51.7%	+21.1	36.7%
TCN Predictive	72.1%	82.6%	+30.5	59.7%

Controller	Alignment	Low-PAC Targeting	PAC Gap ($\times 10^{-6}$)	Stim %
Oracle (upper bound)	100.0%	100.0%	+33.3	48.3%

Metric definitions: Alignment = (Low-PAC Stim Rate + High-PAC Rest Rate) / 2, measuring balanced accuracy of therapeutic targeting. Low-PAC Targeting = percentage of below-median PAC windows where the controller stimulates ("did we treat when the brain needed it?"). PAC Gap = mean PAC during rest minus mean PAC during stim (positive means the controller correctly targets low-PAC periods).

Statistical significance (TCN vs Reactive):

- Alignment: Hedges' $g = +1.31$, $p < 0.001$
- Low-PAC targeting: Hedges' $g = +4.47$, $p < 0.001$
- PAC gap: Hedges' $g = +1.57$, $p < 0.001$
- All Wilcoxon signed-rank tests, bootstrap 95% confidence intervals

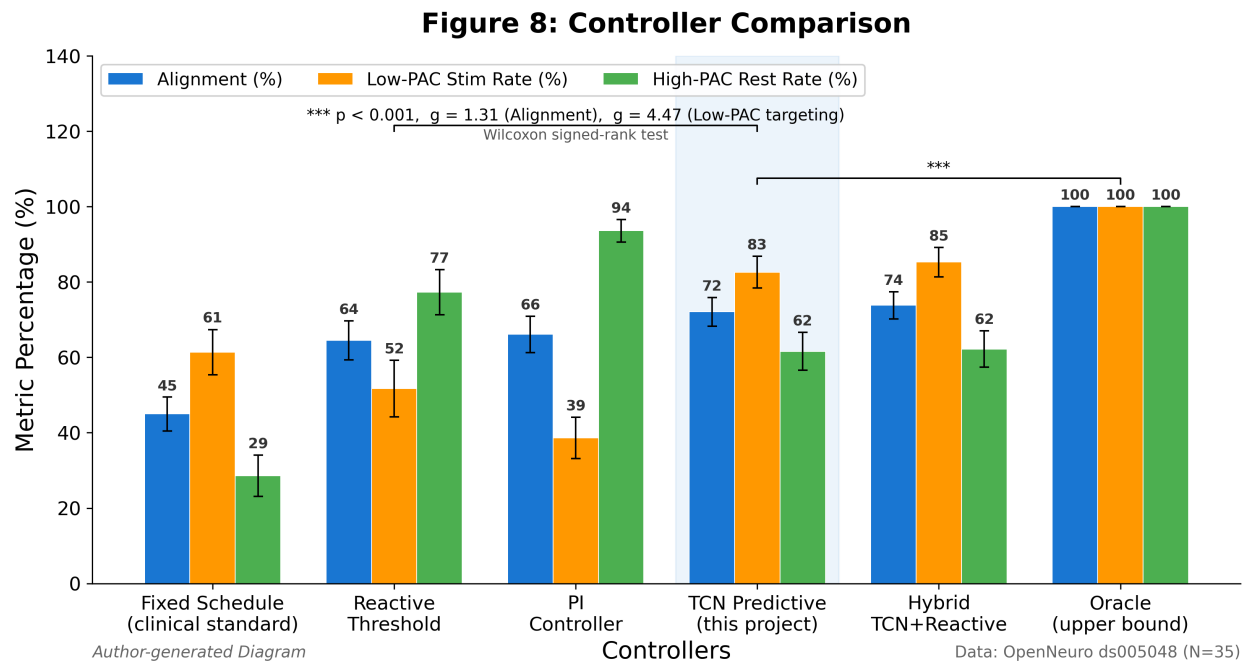
Key findings:

The fixed schedule was not merely weaker – its PAC gap went in the wrong direction (-6.6), meaning it stimulated more during high-PAC than low-PAC periods. Timing was uncorrelated with brain state.

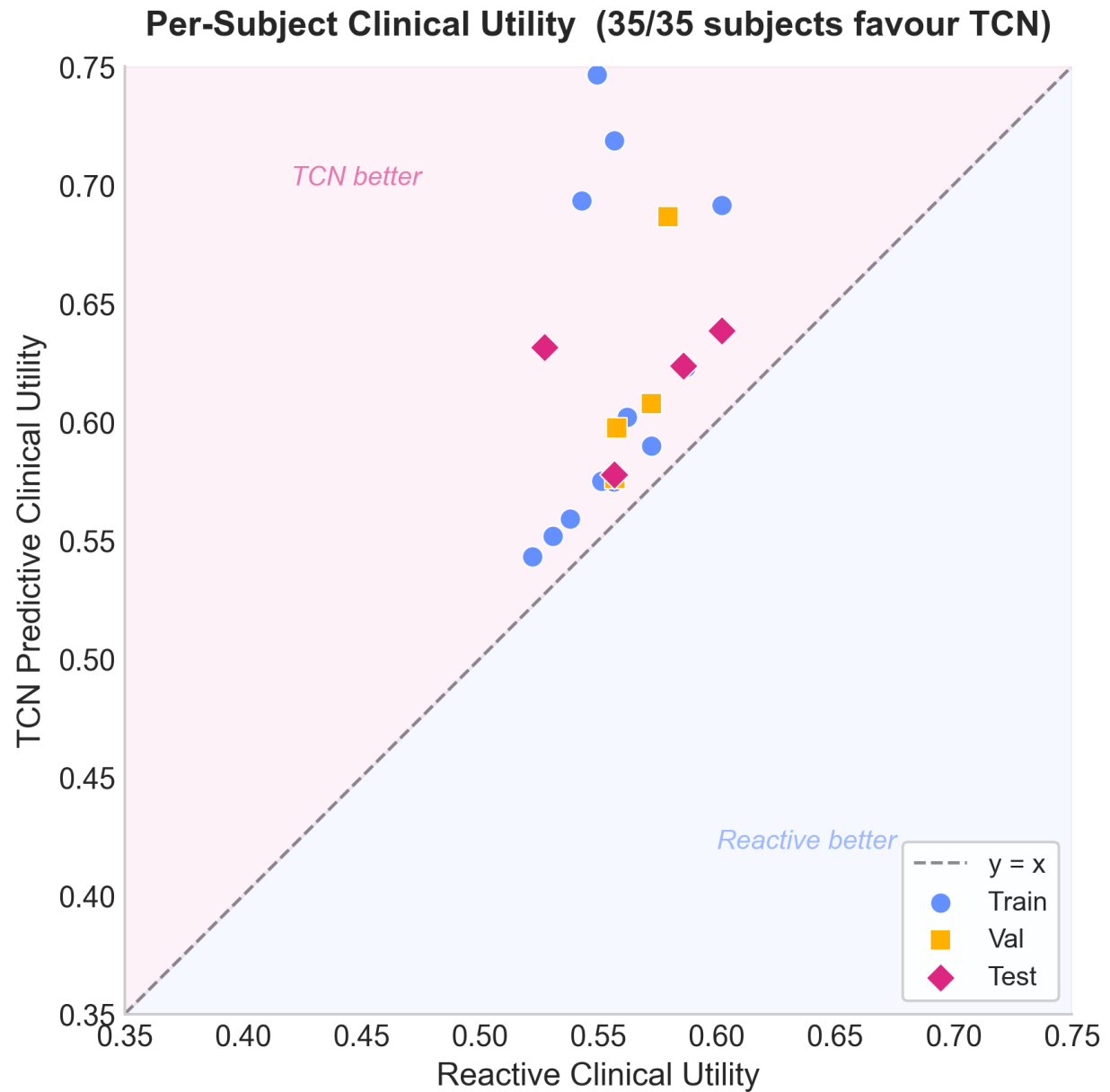
The TCN achieved 92% of oracle performance: PAC gap 30.5 versus 33.3 theoretical maximum ($30.5/33.3 = 92\%$). It targeted 82.6% of low-PAC windows versus Reactive's 51.7% – a 60% improvement in therapeutic precision. And it used less stimulation than Fixed Schedule (59.7% vs 66.6%), meaning better outcomes with less total stimulation.

The reactive controller missed 48% of low-PAC windows that needed treatment. Its conservatism (36.7% total stim) meant high specificity but terrible sensitivity.

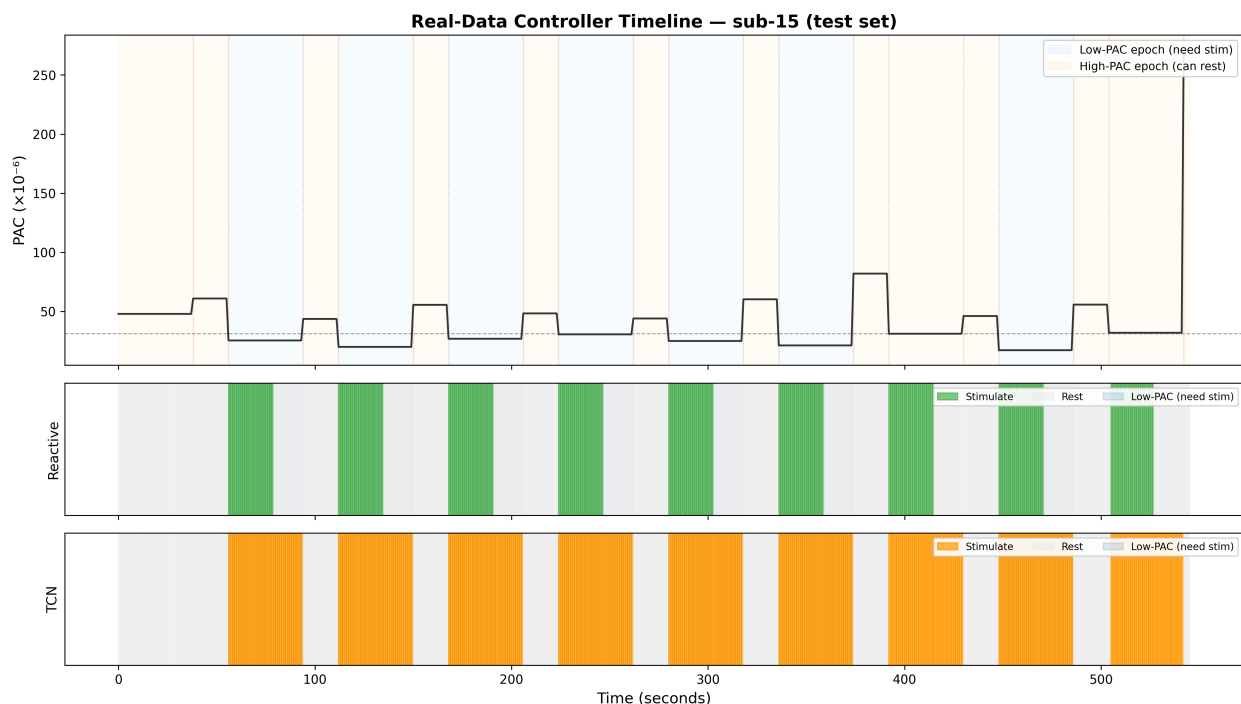
All 35 of 35 subjects showed higher clinical utility with TCN versus Reactive (binomial $p < 0.001$). This held for the 6 held-out test subjects never seen during training, demonstrating robust cross-subject generalization.



Controller comparison on all 35 subjects' real EEG recordings. TCN Predictive outperforms all alternatives on alignment, low-PAC targeting, and PAC gap.



Every patient benefits: 35/35 subjects show higher utility with TCN vs Reactive.



Example controller timeline for one subject, showing how the TCN-driven controller aligns stimulation with periods of low PAC.

Section 9: March 1, 2026 – Documentation Freeze and Submission

March 1, 2026: The science was finished, so today was about packaging. I compiled all results into the poster, abstract, and this notebook for the Synopsys Science Fair submission. The most important discipline was not adding anything new – just consolidating what was validated and making it presentable for judges.

Summary of the complete pipeline:

The system works in two stages. EEGNet (1,457 parameters) estimates current PAC from a 2-second raw EEG window. The Causal TCN (31,043 parameters) then ingests 20 seconds of 73-feature history and forecasts PAC 5 seconds ahead. A personalization module adapts to each patient's individual PAC baseline using a rolling z-score, and the controller uses the TCN's prediction to make proactive stimulation decisions with a 5-second hysteresis.

Conclusions, verified against the poster and validated results:

1. At 5-10 second horizons, the TCN maintains R^2 around 0.25 while all baselines collapse below zero – a $+0.5$ R^2 margin at the operationally relevant range for proactive control.
2. On real patient EEG: 72.1% alignment versus 64.5% reactive ($p < 0.001$), targeting 82.6% of low-PAC windows versus 51.7% – a 60% improvement in therapeutic targeting.
3. All 35 patients benefited, including 6 held-out test subjects. Generalizes across individual EEG patterns.
4. Adaptive advantage increases with fatigue (+9.0% to +11.2%) and holds across four fatigue model assumptions (Hedges' $g = 1.21$ -3.66, all $p < 0.001$).

5. Half of patients habituate while half do not, validating the need for personalized rather than fixed scheduling.

Limitation: Real-data validation uses offline replay on recorded EEG, not live closed-loop streaming. The system makes decisions on real brain data but cannot observe the brain's response to those decisions.

Further research directions:

- Deploy with live EEG streaming for real-time crossover validation (adaptive vs fixed within same session)
- Record 30-60 minute sessions to capture the full habituation time course
- Replace the heuristic controller with reinforcement learning for long-horizon optimization
- Extend to multi-biomarker control (PAC + spectral power + connectivity)

Equipment and Materials

- **Compute:** MacBook Pro (Apple M1 Pro, 16 GB RAM). All computation on local hardware, no cloud GPU required.
- **Software:** Python 3.13, PyTorch, NumPy, SciPy, scikit-learn, h5py, MNE-Python. Git version control.
- **Dataset:** OpenNeuro ds005048 (Lahijanian et al., 2024). 35 dementia patients, 7 frontal EEG channels, 250 Hz. Alternating stimulus (40 Hz AM auditory) and rest epochs. 17,283 two-second windows. Subject-level splits: 24 train / 5 val / 6 test. All data used under open access license.

References

- Iaccarino, H. F., et al. (2016). Gamma frequency entrainment attenuates amyloid load and modifies microglia. *Nature*, 540(7632), 230-235.
- Martorell, A. J., et al. (2019). Multi-sensory γ stimulation ameliorates Alzheimer's-associated pathology and improves cognition. *Cell*, 177(2), 256-271.
- Tort, A. B., et al. (2010). Measuring phase-amplitude coupling between neuronal oscillations of different frequencies. *Journal of Neurophysiology*, 104(2), 1195-1210.
- Lawhern, V. J., et al. (2018). EEGNet: A compact convolutional neural network for EEG-based brain-computer interfaces. *Journal of Neural Engineering*, 15(5), 056013.
- Lahijanian, M., et al. (2024). Auditory γ -band entrainment enhances default mode network connectivity in dementia patients. *Scientific Reports*, 14, 13153.
- Thompson, R. F., & Spencer, W. A. (1966). Habituation: A model phenomenon for the study of neuronal substrates of behavior. *Psychological Review*, 73(1), 16-43.

Submitted to Synopsys Science and Engineering Fair 2026 Category: Biological Science and Engineering, Computational Biology and Bioinformatics